Developer Guide (Part 4): Data Engineering & Feature Engineering for Quantitative Trading

Version: 1.0
Audience: Software Engineers, Quant Developers, Data Engineers, ML Engineers

Prerequisites:

- Part 1 – Introduction & System Overview
- Part 2 – Financial Markets & Market Data Fundamentals
- Part 3 – Mathematical Foundations

Table of Contents

---

1. Purpose

A quantitative trading strategy is only as reliable as the data that powers it.

Before any model is trained or any trade is executed, market data must be:

- Collected
- Validated
- Cleaned
- Normalized
- Stored
- Transformed into useful features

This page explains how production-grade quant platforms build reliable data pipelines that support research, backtesting, and live trading.

---

2. What is Data Engineering?

Data engineering is the process of building systems that collect, process, store, and serve data efficiently.

In quantitative trading, the data engineering layer ensures that:

- Market data arrives with minimal latency.
- Historical data is complete and accurate.
- Data is transformed into reusable features.
- Research and production systems use consistent datasets.

Without robust data engineering, even the most sophisticated trading model can produce unreliable results.

---

3. Market Data Sources

Quantitative trading systems consume data from multiple sources.

Source	Data Provided	Typical Use
Stock Exchanges	Trades, quotes, order book	Live trading
Broker APIs	Market data, account information	Order execution
Financial Data Vendors	Historical prices, fundamentals	Backtesting

News Providers	Financial news, events	Sentiment analysis
Macroeconomic Sources	Interest rates, CPI, GDP	Macro strategies
Alternative Data	Satellite images, social sentiment, web traffic	Alpha generation

> **Note:** Professional trading firms often combine multiple vendors to improve data quality and reduce dependency on a single provider.

---

4. Market Data Types

Historical Data

Used for:

- Backtesting
- Research
- Machine learning model training

Example

Date	Open	High	Low	Close	Volume
2026-01-01	100	105	99	104	1,250,000

Real-Time Data

Delivered continuously during market hours.

Examples:

- Last traded price
- Best bid
- Best ask
- Volume
- Tick updates

Tick Data

Records every market event.

Example

09:15:00.125 BUY   250 101.25
09:15:00.150 SELL 100 101.20
09:15:00.180 BUY   500 101.30

Tick data is essential for:

- High-frequency trading
- Market microstructure analysis
- Order flow models

Order Book Data

Shows active buy and sell orders.

SELL
101.30   500
101.25   300

101.20

101.15   400
101.10   700
BUY

Used by:

- Market making
- Liquidity analysis
- Execution algorithms

---

5. Data Ingestion Pipeline

The ingestion layer collects market data from various providers.

Typical workflow

Exchange APIs
Broker APIs
News APIs
Alternative Data

Responsibilities

- Connect to data providers
- Handle authentication
- Retry failed requests
- Timestamp incoming records
- Buffer bursts of incoming data
- Preserve message ordering where required

---

6. Data Validation

Incoming market data should never be trusted blindly.

Validation checks include

Missing Values

Example:

Price = NULL

Should be rejected or repaired.

Invalid Prices

Examples:

Negative price
Price = -120
Volume = -500

Such records indicate data corruption.

Duplicate Records

Duplicate ticks distort backtesting results.

Validation should detect duplicate:

- Timestamp
- Symbol
- Price
- Quantity

Timestamp Validation

Ensure records arrive in chronological order.

Out-of-order data may indicate:

- Network delay
- Provider issues
- Clock synchronization problems

---

7. Data Cleaning

Typical cleaning steps:

- Remove duplicates
- Fill missing values

Methods include:

- Forward Fill (FFill)
- Backward Fill (BFill)
- Interpolation

Adjust Historical Prices

Adjust for:

- Stock splits

- Dividends
- Bonus issues

Failure to adjust historical prices leads to incorrect performance calculations.

Normalize Symbols

Example:

AAPL
Apple
NASDAQ:AAPL

All should map to one canonical identifier.

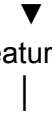
---

8. Data Storage Architecture

Production systems often separate storage into multiple layers.

Raw Data

Validated Data

Clean Data

Feature Store

Research Database

Common Storage Technologies

Layer	Technology
Raw Data	Object Storage (S3, Azure Blob)
Time Series	ClickHouse, InfluxDB

Relational	PostgreSQL
Analytics	Snowflake
Cache	Redis

---

9. ETL Pipeline

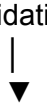
ETL stands for:

- Extract
- Transform
- Load

Example

Exchange

Extract

Validation

Cleaning

Transformation

Database

Typical transformations

- Time zone conversion
- Currency normalization
- Corporate action adjustments
- Feature calculation

---

10. Feature Engineering

Feature engineering converts raw market data into informative variables that trading strategies or machine learning models can use.

Example

Raw Data

Open
High
Low
Close
Volume

↓

Features

20-day SMA
RSI
ATR
Daily Return
Rolling Volatility

---

11. Technical Indicator Features

Common engineered features include:

Indicator	Purpose
SMA	Trend
EMA	Short-term trend
RSI	Overbought/Oversold
MACD	Momentum
ATR	Volatility
Bollinger Bands	Price deviation
VWAP	Average traded price

These indicators transform raw prices into more meaningful signals.

---

12. Statistical Features

Examples:

- Daily Return
- Rolling Mean
- Rolling Standard Deviation
- Z-Score
- Correlation
- Beta
- Skewness
- Kurtosis

These features help quantify market behavior beyond simple price movements.

---

13. Machine Learning Features

Machine learning models often require a richer feature set.

Lag Features

Close(t-1)
Close(t-2)
Close(t-5)

Rolling Window Features

Mean(20)
Std(20)
Max(20)
Min(20)

Calendar Features

- Day of Week
- Month
- Quarter
- Holiday Indicator

Cross-Asset Features

- Index Return
- Sector Return
- Currency Movement

---

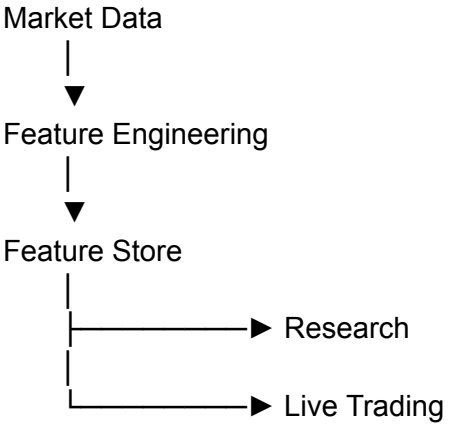
14. Feature Store

A feature store is a centralized repository for engineered features.

Benefits

- Eliminates duplicate feature calculations
- Ensures consistency between research and production
- Improves reproducibility
- Simplifies model deployment

Example



---

15. Data Versioning

Financial datasets evolve over time.

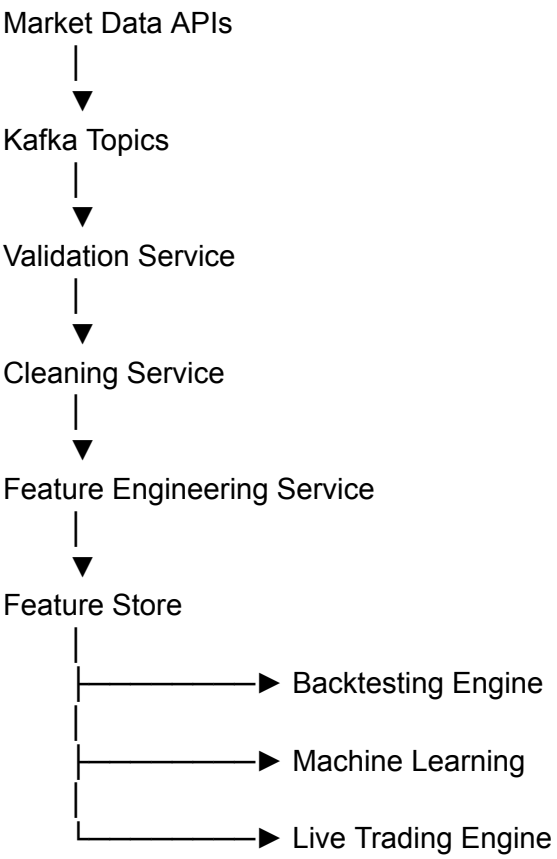
Best practices

- Version datasets
- Track schema changes
- Record corporate action adjustments
- Store metadata
- Make experiments reproducible

---

16. Production Architecture

A modern data pipeline might look like this:



This architecture allows each stage to scale independently and supports both offline research and real-time trading.

---

17. Best Practices

- Treat data as a product—monitor its quality continuously.
- Keep raw data immutable for auditing and reprocessing.
- Separate raw, validated, and engineered datasets.
- Automate validation and cleaning pipelines.
- Version features and datasets for reproducibility.
- Ensure the same feature calculations are used in both backtesting and live trading.
- Monitor pipeline latency and data freshness.
- Document every transformation applied to the data.

Summary

Data engineering and feature engineering are foundational to quantitative trading. A well-designed pipeline ensures that strategies operate on clean, consistent, and timely data, while feature engineering transforms raw market information into meaningful signals for statistical models and machine learning algorithms. Investing in robust data infrastructure improves research quality, reduces operational risk, and increases confidence in production trading systems.

References

Books

- Ernest P. Chan — *Algorithmic Trading*
- Ernest P. Chan — *Quantitative Trading*
- Marcos López de Prado — *Advances in Financial Machine Learning*
- Yves Hilpisch — *Python for Algorithmic Trading*

Research Papers

- Yang et al. — *Qlib: An AI-oriented Quantitative Investment Platform*
- Liu et al. — *FinRL: A Deep Reinforcement Learning Library for Automated Stock Trading*